

MixBytes()

[SECURITY AUDIT REPORT]

Gearbox Protocol Integrations-v3 (Midas RWA)

PREPARED FOR: GEARBOX

REPORT DATE

August 14, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	6
1.4 Security Audit Methodology	9
1.5 Risk Classification	11
1.6 Summary of Findings	12
2. Findings Report	12
2.1 Critical	12
2.2 High	13
H-1 Canceled redeemer transfer gives the recipient an unaccounted recovery right	13
2.3 Medium	14
M-1 Unguarded feed call propagates external reverts into collateral and liquidation checks	14
M-2 Cancellation can socialize bad debt before a later recovery	15
2.4 Low	16
L-1 Unconditional mToken sweep can block quote-token withdrawal	16
L-2 Live-rate valuation can exceed a saved-rate payout	17
L-3 Zero-output redemption requests create live redeemers outside normal tracking	18
3. About MixBytes	19

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Gearbox users hold leveraged positions through Credit Accounts. The audited suite connects those accounts to Midas RWA products through instant and delayed flows. The instant issuance and redemption adapters let a Credit Account exchange supported quote tokens and mTokens directly against Midas vaults while enforcing the limits supplied through the Gearbox multical.

Delayed redemption is coordinated by `MidasGateway`. Each request receives a fresh `MidasRedeemer` clone, which holds the mToken, submits the upstream request and later receives either the quote-token payout or returned mToken. While settlement is pending, `MidasRedemptionVaultPhantomToken` represents the expected quote-token amount as collateral. Once assets arrive, the gateway lets the owning Credit Account withdraw them from the clone.

The gateway also integrates Midas eligibility requirements with Gearbox account checks. During full liquidation, `MidasLiquidator` temporarily opens a transfer window so an unsettled redeemer can move from the liquidated account to a permitted recipient. The resulting security boundary spans Gearbox collateral accounting and liquidation, gateway ownership bookkeeping, immutable redeemer clones, and the external Midas vault, data feed and access-control state.

Two security auditors reviewed the integration independently and then consolidated their results. The review combined threat modelling, line-by-line inspection of the twelve in-scope contracts, complete call-path tracing through Gearbox core and the Midas vaults, and claim-by-claim refutation of both manual and scanner-generated candidates.

During the re-audit, commit `38ee0623547f9d0df65bddea62e26751c9bae1a7` introduced three targeted remediations. A new Midas-specific aliased loss policy blocks loss liquidations while an account has a rejected redemption request. Pending claims can now be valued using the initial mToken rate stored in the request instead of the live data feed. The residual mToken sweep is also best-effort, so a failed mToken transfer no longer blocks quote-token withdrawals.

Together, these changes prevent cancellation-driven losses from being realized before a potential recovery, remove live-feed dependency and live-versus-saved-rate valuation mismatches when initial-rate pricing is selected, and decouple quote-token withdrawals from mToken transfer availability. No new security issues were identified in the reviewed changes.

Scope Limitations

The audit scope is limited to the listed `integrations-v3` contracts. Gearbox core, the Midas contracts, the previous integration version, tests, mocks and SDK were inspected only where their behavior determines whether an in-scope path is reachable or harmful.

Review Coverage

In addition to the audit team's general smart-contract security checklist, the review covered the following integration-specific questions:

- Can a pending claim always be valued, withdrawn or transferred without making the Credit Account unmanageable?
- Does collateral valuation remain conservative across live-rate, saved-rate, cancellation and settlement transitions?
- Can full and partial liquidation realize every asset that was included in the account's collateral value?
- Do Midas greenlist, blacklist and pause actions preserve safe exit and liquidation paths?
- Can the gateway's ownership and pending sets diverge from the core enabled-token mask in a way that reallocates value?
- Are decimals, rounding, slippage limits, fee handling and external-call failure modes consistent with the production Midas paths?

Additional Attack Hypotheses

In addition to the issues described in this report, the audit examined the following protocol-specific attack hypotheses.

- **Stranded collateral on the liquidation helper.** A successful liquidation cannot consume less collateral than `MidasLiquidator` pulls: the helper, facade and Credit Manager use the same exact amount. Duplicate entries can revert, but atomicity prevents a successful partial spend or stranded remainder.
- **Decimal mismatch weakening instant-issuance slippage.** `rateMinRAY` expresses output base units per input base unit. Under that convention, converting the Midas input to 18 decimals while keeping the `mToken` minimum in native 18-decimal units produces the intended limit.
- **Rounding divergence between pending valuation and payout.** Gearbox's nested-floor conversion and the Midas transfer path produce the same native-unit payout at the same rate. Rate changes between request and settlement are covered separately and do not create a rounding exploit.
- **Zero-amount delayed redemption.** Both target vaults reject zero input before storing a request. The revert atomically rolls back clone registration, role grants, approvals and transfers.
- **Pending value making the generic withdraw-all sentinel revert.** The phantom intentionally includes unsettled value that cannot yet be paid. Exact claimable-amount and per-redeemer withdrawal routes remain available, so this is not a loss or lock.
- **A fake Credit Account draining the liquidation helper.** Caller-supplied contracts can make the helper approve a caller-selected spender, but only after pulling the same amount from that caller. The production success path leaves no helper balance to steal.
- **A zero request record causing a persistent division-by-zero freeze.** The transient default record cannot survive the outer transaction, and the target vault stores non-zero request rates before returning. No production callback can turn the temporary state into a lasting account failure.

Additional Notes

- Gateway and redeemer dependencies are immutable while the corresponding Midas contracts are upgradeable, so some Midas-side changes require redeploying the integration.
- Removing a borrower from the Midas greenlist does not change Gearbox collateral accounting: existing `mToken` balances and pending or claimable redemption amounts continue to contribute to account health, while borrower-gated gateway operations such as `transferRedeemer` become unavailable. Whether this valuation remains appropriate depends on how Midas treats the offboarded position: if the economic claim is preserved, existing redemption requests must remain settleable; if it is extinguished, Midas must burn any remaining `mToken` balance and cancel pending requests so that their collateral value becomes zero. Already-settled quote-token balances are unaffected by the Midas role and remain claimable.

Overall, the codebase is compact, well structured, and separates integration responsibilities clearly. Following the re-audit, H-1, M-2, L-1 and L-2 are considered fixed, while M-1 and L-3 remain acknowledged. The client's rationale for accepting the residual risk was reviewed and considered justified. The implemented remediations are covered by focused regression tests. The updated revision is considered ready for production and completing final deployment configuration and bytecode verification.

1.3 Project Overview

Summary

CLIENT	Gearbox Protocol
CATEGORY	Lending
PROJECT	integrations-v3 (Midas RWA)
TYPE	Solidity
PLATFORM	EVM
TIMELINE	06.08.2026 – 14.08.2026

Scope of Audit

```
contracts/integrations/common/CACheckerTrait.sol
```

```
contracts/integrations/common/RedemptionLogger.sol
```

```
contracts/integrations/common/RedemptionLoggingTrait.sol
```

```
contracts/integrations/midas/MidasGateway.sol
```

```
contracts/integrations/midas/MidasRedeemer.sol
```

```
contracts/integrations/midas/MidasGatewayAdapter.sol
```

```
contracts/integrations/midas/MidasLiquidator.sol
```

```
contracts/integrations/midas/MidasRedemptionVaultPhantomToken.sol
```

```
contracts/integrations/midas/MidasIssuanceVaultAdapter.sol
```

```
contracts/integrations/midas/MidasRedemptionVaultAdapter.sol
```

```
contracts/integrations/midas/MidasDegenNFT.sol
```

```
contracts/integrations/midas/MidasDecimals.sol
```

```
contracts/integrations/midas/MidasAliasedLossPolicyV3.sol
```

Versions Log

DATE	COMMIT HASH	NOTE
06.08.2026	e56429547af7f9452d5d552b419d07079d2695c4	Initial Commit
12.08.2026	38ee0623547f9d0df65bddea62e26751c9bae1a7	Re-audit Commit
14.08.2026	22754fff8278c5338e5ff0249d9c37d4680186f4	Minor <code>useSafePrices</code> Fix Commit

Mainnet Deployments

Deployment verification will be conducted via <https://bcr.gearbox.finance/>

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

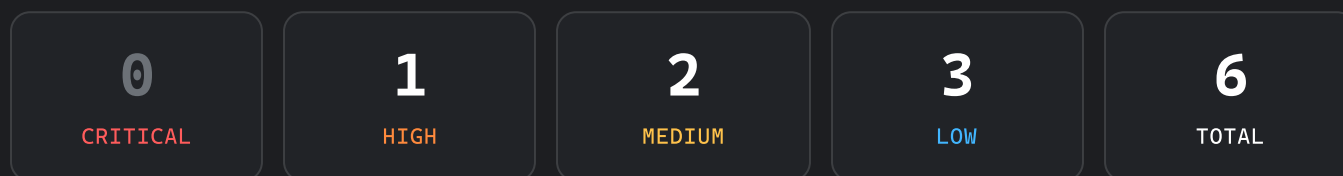
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
H-1	Canceled redeemer transfer gives the recipient an unaccounted recovery right	● HIGH	● FIXED
M-1	Unguarded feed call propagates external reverts into collateral and liquidation checks	● MEDIUM	● ACKNOWLEDGED
M-2	Cancellation can socialize bad debt before a later recovery	● MEDIUM	● FIXED
L-1	Unconditional mToken sweep can block quote-token withdrawal	● LOW	● FIXED
L-2	Live-rate valuation can exceed a saved-rate payout	● LOW	● FIXED
L-3	Zero-output redemption requests create live redeemers outside normal tracking	● LOW	● ACKNOWLEDGED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

H-1 Canceled redeemer transfer gives the recipient an unaccounted recovery right

● HIGH

● FIXED AS OF 38ee0623

DESCRIPTION

`MidasGateway.sol:192-208` allows a liquidation to transfer any redeemer still in the pending set without checking its request status. A canceled redeemer has zero phantom value under `MidasRedeemer.sol:107-116`, but control over it can still represent the right to receive a later Midas recovery. The recipient receives that right for no consideration and the former owner loses it.

Cancellation and compensation are separate administrative operations on the Midas side.

`RedemptionVault.sol:378-386` changes the request to `Canceled` without returning assets, while `ManageableVault.sol:189-197` lets a vault administrator later choose the token, amount and recipient of a withdrawal. The redeemer remains transferable between those transactions until another gateway call prunes it from the pending set.

This sequence is observable on the mfONE vault. Requests `#4`, `#10` and `#47` were canceled in transactions `0xd3c2...23a`, `0x2636...618e` and `0xea4e...3c70`. Separate administrator transactions `0x848f...833f`, `0x5f1c...88d9` and `0xd567...f24` later returned mfONE tokens to the respective request senders.

If the request sender is a Gearbox redeemer, the returned mToken lands on that clone.

`MidasRedeemer.sol:93-102` then sends it to the account currently recorded on the clone, which `transferRedeemer` has replaced with the recipient. The former owner loses the entire recovery even though none of it was included in the liquidation accounting.

The impact can be High because the transfer can reallocate the full recovery right, and the mfONE cancellation shows that the amount can exceed 1% of a user's position. The likelihood is Medium: There're observations of cancelling a request by Midas, which could easily lead the account to be insolvent before the separate return.

RECOMMENDATION

We recommend prohibiting `transferRedeemer` when the underlying request is in the `REJECTED` (`Canceled`) state.

CLIENT'S COMMENTARY

Fixed in `38ee0623547f9d0df65bddea62e26751c9bae1a7`. The new `MidasAliasedLossPolicyV3` prevents lossy liquidations if there are any rejected redeemers in the pending set.

2.3 Medium

M-1 Unguarded feed call propagates external reverts into collateral and liquidation checks

● MEDIUM

● ACKNOWLEDGED

DESCRIPTION

The unguarded call at `MidasRedeemer.sol:107-116` lets a Midas feed revert propagate through the phantom token's `balanceOf`. Gearbox core reads that balance before weighting it at `CollateralLogic.sol:95-108`, so ordinary collateral and liquidation checks fail while the account has an enabled phantom token for a `PENDING` request.

The upstream feed reverts when its answer is stale or outside configured bounds at `DataFeed.sol:156-174`. The target configurations make this reachable without malformed input: mGLOBAL has a 60-day health window against observed update gaps of 24-45 days, while mfONE has a 30-day window and bounds that a permitted administrative update can cross.

While the feed is failing, collateral checks that reach the enabled phantom token revert. The borrower can still remove it by recapitalizing or repaying debt and zeroing its quota in the same multicall, because the final collateral check no longer reads that token.

Liquidation remains blocked while any request is `PENDING`, as collateral is evaluated before the liquidator's multicall. Restoring the feed or transitioning the request out of `PENDING` resolves the lock; normal settlement atomically marks it `Processed`, after which valuation no longer reads the feed. The impact is therefore temporary valuation and liquidation unavailability, not a lock of settled funds.

The impact is Medium because Gearbox cannot restore collateral valuation or liquidate an affected unhealthy account through any protocol-controlled recovery path. Borrower recapitalization is voluntary, while restoring the feed or transitioning the request requires action from the external Midas protocol. Given the plausible feed-staleness and administrative triggers, likelihood is Medium. The resulting severity is Medium.

RECOMMENDATION

We recommend handling feed failures inside `pendingTokenOutAmount` by returning zero or using the stored request rate.

CLIENT'S COMMENTARY

Acknowledged. In most cases feed failure is temporary and is related to the feed going stale. At the same time, falling back to initial rate may cause invalid liquidations of Credit Accounts for mTokens that use the current NAV rate for settling. We believe this is a worse problem than balances being unavailable for a time, and would prefer to keep the current logic.

Note also that even if the feed is unavailable, the problem will resolve if Midas accepts or rejects the corresponding request.

M-2 Cancellation can socialize bad debt before a later recovery

● MEDIUM

● FIXED AS OF 38ee0623

DESCRIPTION

When Midas cancels a request, `MidasRedeemer.sol:107-116` immediately reports zero even though Midas may later return the escrowed mToken in a separate administrator transaction. A leveraged Credit Account can therefore move from solvent to bad debt before the recovery arrives.

Full liquidation realizes that loss against the pool. `CreditManagerV3.sol:321-339` calculates the loss, transfers the available underlying and calls `repayCreditAccount`. `PoolV3.sol:484-519` then burns treasury shares to cover the loss, emits any uncovered portion and reduces expected liquidity. The Credit Manager removes the account's quotas and resets its enabled-token mask but deliberately leaves the borrower unchanged. The canceled redeemer also remains assigned to that account unless the liquidator transfers it. If Midas later sends mToken to the clone, the borrower can use `withdrawFromRedeemer(redeemer, 0)` to sweep the recovery to the same Credit Account; if the redeemer was transferred, the chosen recipient receives it instead. Neither path repays the pool that absorbed the shortfall.

Restricting loss liquidation in core is insufficient unless the wrapper preserves the real caller. `MidasLiquidator.sol:43-67` is permissionless and calls the facade as itself. A permissioned loss policy therefore authenticates the helper instead of the external liquidator; granting the helper `LOSS_LIQUIDATOR` would let any caller use that role.

The impact can be High because the pool may realize the full canceled position as bad debt while the corresponding recovery later accrues elsewhere. Likelihood is Low because Midas must cancel and later compensate the request while the account is sufficiently leveraged to incur a loss. The resulting severity is Medium.

RECOMMENDATION

We recommend allowing bad-debt liquidation of canceled requests only for authenticated external callers and recording the pool as beneficiary of any later recovery before the loss is realized.

CLIENT'S COMMENTARY

Fixed in `38ee0623547f9d0df65bddea62e26751c9bae1a7`. The new `MidasAliasedLossPolicyV3` prevents lossy liquidations if there are any rejected redeemers in the pending set.

2.4 Low

L-1 Unconditional mToken sweep can block quote-token withdrawal

● LOW

● FIXED AS OF 38ee0623

DESCRIPTION

`MidasRedeemer.sol:93-102` sweeps the redeemer's entire mToken balance after every quote-token withdrawal. If that mToken cannot be transferred to the recorded account, the whole call reverts and the unrelated quote-token payment remains on the clone.

A third party can send mToken to a pending clone before Midas settles the request. Settlement burns the requested amount at `RedemptionVault.sol:525-540` but does not consume the donated balance. If Midas later pauses the token or restricts the recorded account, withdrawing the settled quote token also attempts the failing mToken transfer. During liquidation, the same sweep can increase the account's mToken balance and fail the core post-call balance check unless the liquidator removes that mToken in the same multical.

RECOMMENDATION

We recommend making the mToken sweep optional so that quote-token withdrawal never depends on transferring a second asset.

CLIENT'S COMMENTARY

Fixed in `38ee0623547f9d0df65bddea62e26751c9bae1a7` by making mToken transfers optional.

L-2 Live-rate valuation can exceed a saved-rate payout

● LOW

● FIXED AS OF 38ee0623

DESCRIPTION

`MidasRedeemer.sol:107-116` values a pending request with the live mToken feed even though the request already stores its submission rate. The target vault can settle at that stored rate through `RedemptionVault.sol:327-340`, so a rising live rate can make the phantom balance exceed the eventual payout. Safe approval can also use another rate within `variationTolerance`, while the unrestricted administrator path has no enforceable lower bound.

The discrepancy is concrete: stored mfONE request rates lagged the live rate by 0.43–0.62% in the reviewed snapshot. It temporarily overstates collateral, although an end-to-end loss to the pool was not established.

RECOMMENDATION

We recommend valuing a pending request at the lower of its live and stored mToken rates.

CLIENT'S COMMENTARY

Fixed in `38ee0623547f9d0df65bddea62e26751c9bae1a7`. Most mTokens settle redemptions based on the current NAV rate. However, there are some (e.g., mfONE) that use the initial mTokenRate at the request time for settling. We've added a constructor parameter that determines which rate is used to value pending redemptions.

L-3 Zero-output redemption requests create live redeemers outside normal tracking

● LOW

● ACKNOWLEDGED

DESCRIPTION

Every delayed redemption creates a dedicated redeemer clone and submits a Midas request before the integration checks whether the projected quote-token payout is economically nonzero.

`pendingTokenOutAmount()` can therefore return zero for a newly created request that is still live when its projected payout rounds down during conversion from 18-decimal accounting to the quote token's native decimals.

When a successful aggregate withdrawal later traverses such a redeemer, `MidasGateway.withdraw()` interprets the zero value as a terminal signal and removes it from the pending set. The result is a live but economically empty Midas request represented only by the gateway's ownership record: it no longer contributes to the phantom balance and is not returned by `pendingRedeemers()`. The Credit Account still owns the redeemer and can recover a later payout through `withdrawFromRedeemer()`, so the immediate omitted value is dust and no loss of funds is claimed. Nevertheless, the integration unnecessarily creates a clone and an external request that then falls outside its normal lifecycle tracking.

RECOMMENDATION

We recommend rejecting a delayed redemption if its projected quote-token payout rounds to zero, reverting the clone and Midas request. For accepted requests, remove the redeemer only when the request is terminal and its claimable balance is zero.

CLIENT'S COMMENTARY

Acknowledged. The issue is insignificant in practice, since most mTokens have minimal investment caps (as well as Gearbox Credit Facades having minimal borrowing caps). At the same time, any damage can be only self-inflicted by the user (by making an extremely small request deliberately). Preventing this on UI side is sufficient.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io